Actividad práctica 4

Implementación del núcleo de negocio · TechStore U4

Nombres : Equipo 8 Andrés Caleb Leija Pedroza David Alonso Torres Guerrero Emir Misael López Pérez	**Matrículas**: 2144830 2144813 2150561
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Modulo Netacad: 5 en Adelante	**Actividad:** Informe de Arquitectura
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.

1. Introducción

Este reporte documenta la Actividad práctica 4 del curso Componentes de Lógica de Negocio (FIME, UANL). El objetivo es implementar el núcleo de negocio de una tienda ficticia, TechStore, y demostrar mediante pruebas que sus reglas funcionan y que los datos inválidos se rechazan.

El programa lee clientes, productos y pedidos desde archivos CSV y los convierte en objetos de dominio. Después aplica reglas de descuento con el patrón Specification, controla el ciclo de vida de cada pedido con una máquina de estados que emite Domain Events y expone sus operaciones mediante una API REST hecha con FastAPI. Los datos viven en memoria: al cerrar el programa los cambios se pierden y no se usa base de datos.

El documento sigue el orden de la práctica. Primero se describen el material recibido, la preparación del ambiente y la lectura de los datos; luego el dominio, las Specifications, la máquina de estados, los 15 escenarios, la API y las pruebas. Al final se incluyen la lista de verificación, las respuestas a las preguntas del reporte y las reflexiones del equipo. Los cuatro entregables técnicos de la actividad se encuentran en las secciones siguientes:

Entregable técnico	Sección
Entidades y Value Objects con invariantes	5. Dominio: entidades y Value Objects
Reglas complejas con Specification Pattern (AND, OR, NOT)	7. Specification Pattern
Máquina de estados con Domain Events	8. Máquina de estados y Domain Events
API REST con manejo de excepciones	10. API REST con FastAPI

2. Material recibido

El paquete del profesor contiene los datos de la tienda y un cargador que depende de las clases que se implementan en la práctica.

Archivo	Contenido
datos/clientes.csv	12 clientes. es_vip y activo: 1 = sí, 0 = no.
datos/productos.csv	12 productos, con precio_mxn y existencia.
datos/pedidos.csv	16 pedidos; todos comienzan en estado DRAFT.
datos/detalle_pedidos.csv	21 renglones de productos por pedido.
datos/escenarios.csv	15 casos (E01 a E15) que se deben ejecutar.
cargar_csv.py	Cargador de datos; importa las clases del dominio.
README.txt	Reglas del paquete y guía de uso.

Dos aclaraciones del material: el campo existencia es solo información del catálogo, porque la práctica no exige descontar inventario; y el límite de 20 corresponde a la suma de unidades del pedido, no al número de renglones.

3. Preparación del ambiente

El proyecto requiere Python 3.11 o superior. Se trabajó dentro de un entorno virtual (.venv) y se instalaron las dependencias con pip; no se necesitan permisos de administrador.

```
python --version
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
python -m pip install fastapi uvicorn pytest httpx
```


unidos por un cambio transformador

La estructura del proyecto es la siguiente. La carpeta app contiene el dominio, las Specifications y la API; tests contiene las pruebas, y datos conserva los cinco CSV sin modificar.

```
techstore-u4/
  datos/               (cinco archivos CSV)
  cargar_csv.py
  README.txt
  app/
    __init__.py
    domain.py
    specifications.py
    api.py
  tests/
    test_domain.py
    test_api.py
```

4. Lectura de los CSV y sus relaciones

Los CSV del paquete se relacionan mediante identificadores; con ellos se reconstruye cada pedido: quién lo hizo, qué productos incluye y cuánto cuesta.

Archivo	Llave	Relación
clientes.csv	cliente_id	Identifica al cliente. es_vip: membresía; activo: puede confirmar pedidos (1 = sí, 0 = no).
pedidos.csv	pedido_id	Identifica el pedido. cliente_id apunta a clientes.csv.
detalle_pedidos.csv	pedido_id, sku	pedido_id enlaza con el pedido y sku enlaza con productos.csv.
productos.csv	sku	Da el precio_mxn de cada producto; existencia es solo información del catálogo.

Cómo se calcula un pedido

El subtotal de un renglón es cantidad × precio unitario. Por ejemplo, si un renglón indica SKU001 con cantidad 2 y productos.csv registra un precio de 600.00, el subtotal es 2 × 600.00 = 1,200.00 MXN. El total del pedido es la suma de sus subtotales, y las unidades son la suma de las cantidades (no el número de renglones).

Actividad corta: cálculo manual

Cliente, SKU y cantidades salen de pedidos.csv y detalle_pedidos.csv; los precios, de productos.csv. El cálculo se hizo antes de escribir código.

O001 · cliente C001 (Ana Torres)

SKU	Producto	Cantidad	Precio unitario	Subtotal
SKU001	Mouse inalambrico	2	600.00	1,200.00
SKU002	Teclado mecanico	2	450.00	900.00
Total				**2,100.00 MXN**

O006 · cliente C006 (Pedro Soto)

SKU	Producto	Cantidad	Precio unitario	Subtotal
SKU008	Laptop empresarial	1	18,000.00	18,000.00
Total				**18,000.00 MXN**

O010 · cliente C010 (Jorge Luna)

SKU	Producto	Cantidad	Precio unitario	Subtotal
SKU003	Cable USB-C	1	120.00	120.00
SKU009	Hub USB	1	380.00	380.00
Total				**500.00 MXN**

Resultado

Pedido	Cliente	Unidades	Total manual
O001	C001 · Ana Torres	4	2,100.00 MXN
O006	C006 · Pedro Soto	1	18,000.00 MXN
O010	C010 · Jorge Luna	2	500.00 MXN

5. Dominio: entidades y Value Objects

El dominio vive en `app/domain.py`. Tiene dos Value Objects (`Money` y `OrderItem`) y dos entidades (`Customer` y `Order`). Los 41 tests de `tests/test_domain.py` pasan.

Value Objects

Son inmutables (`@dataclass(frozen=True)`) y se comparan por valor. Sus invariantes se validan en el constructor y, si se violan, lanzan `InvalidOrderData`.

Clase	Invariantes	Excepción
`Money`	`amount` debe ser `Decimal`. Debe ser finito y ≥ 0. La moneda solo puede ser `MXN`. `add()` rechaza monedas distintas y devuelve un `Money` nuevo.	`InvalidOrderData`
`OrderItem`	`sku` es texto no vacío. `quantity` es entero > 0. El `subtotal` se calcula (`unit_price × quantity`), no se captura a mano.	`InvalidOrderData`

Entidades

Ambas tienen identidad por `id`. `Order` es el Aggregate Root y concentra las reglas del ciclo de vida.

Clase	Papel	Invariantes
`Customer`	Entidad simple con `id`, `is_vip` y `active`.	Sin validaciones propias. La regla de cliente activo la aplica `Order.confirm()`.
`Order`	Aggregate Root con `id`, `customer`, `has_promotion`, `status`, `items` y `events`.	Ver la lista siguiente.

Invariantes de Order

- `units` es la suma de cantidades, no el número de renglones.
- El máximo es 20 unidades por pedido, verificado antes de agregar el artículo.
- `add_item` solo funciona en DRAFT y rechaza monedas distintas de MXN.
- `confirm` exige DRAFT, al menos un artículo y un cliente activo.
- `ship` exige CONFIRMED y una guía no vacía.
- `cancel` exige CONFIRMED y un motivo no vacío.
- `deliver` y `return_order` exigen SHIPPED.
- Si una operación falla, el estado no cambia y no se agrega ningún evento.

Piezas de apoyo

- `DomainEvent`: inmutable, con `name`, `order_id`, `customer_id`, `total`, `detail` y `occurred_at`.
- `OrderStatus`: enum con seis estados (DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, RETURNED).
- Excepciones: `DomainException` → `OrderException` → `InvalidOrderData` (HTTP 422), `InvalidOrderTransition` (HTTP 409) y `OrderItemLimitExceeded` (HTTP 422).

6. Carga de los datos

Primer punto de control: con el dominio implementado, `python cargar_csv.py` carga los 16 pedidos, todos en estado DRAFT. Cada línea muestra el ID, el estado, las unidades y el total; por ejemplo, O001 tiene 4 unidades y 2100.00 MXN, O007 está vacío y O008 tiene 20 unidades.



Figura 1. *Carga de los 16 pedidos con python cargar_csv.py.*

7. Specification Pattern

El zip trae 4 reglas simples y 3 compuestas con AND, OR y NOT. Además se proponen 7 reglas adicionales, que se probaron aparte sobre los 16 pedidos y no forman parte de `specifications.py`.

Reglas que trae app/specifications.py

La base es `Specification` con `&` (AND), `|` (OR) y `~` (NOT). Una Specification solo responde verdadero o falso y no modifica el pedido.

Reglas simples

Specification	Condición
`MinimumAmountSpec(min)`	Total estrictamente mayor que `min` (`>`, no `>=`).
`VIPCustomerSpec`	El cliente es VIP.
`MinimumUnitsSpec(n)`	Suma de unidades ≥ `n` (cuenta unidades, no renglones).
`ActivePromotionSpec`	El pedido tiene promoción activa.

Reglas compuestas

Regla	Composición	Operadores
`TEN_PERCENT`	`MinimumAmount(500) & VIP & MinimumUnits(4) & ~ActivePromotion`	AND, NOT
`FIVE_PERCENT`	`MinimumAmount(5000) & ~ActivePromotion`	AND, NOT
`ANY_DISCOUNT`	`TEN_PERCENT | FIVE_PERCENT`	OR

`discount_rate()` aplica primero el 10 % y luego el 5 %, como pide el paquete.

Reglas adicionales propuestas

Se arman con las mismas piezas y reflejan reglas que hoy están como `if` dentro de `Order` o que se leen en los escenarios.

Regla propuesta	Composición	Para qué sirve
`ActiveCustomerSpec`	`customer.active`	Simple. Es la regla de E03.
`NonEmptyOrderSpec`	`len(items) > 0`	Simple. Es la regla de E06.
`WithinUnitLimitSpec`	`units <= 20`	Simple. Cubre E07 y E08.
`CONFIRMABLE`	`ActiveCustomer & NonEmpty & WithinUnitLimit`	Responde "¿se puede confirmar?" sin lanzar excepciones.
`ONLY_5`	`FIVE_PERCENT & ~TEN_PERCENT`	Pedidos que reciben solo el 5 %.
`VIP_NO_DISCOUNT`	`VIP & ~ANY_DISCOUNT`	VIP que no recibe descuento, como O011.
`BLOCKED_BY_PROMO`	`ActivePromotion & (VIP | MinimumAmount(5000))`	Pedidos que habrían tenido descuento y la promoción lo bloquea, como O005.

Resultado sobre los pedidos

El 10 % y el 5 % coinciden con la bitácora E01–E15. Se muestran los pedidos ligados a un escenario de reglas.

Pedido	VIP	Activo	Promo	Unid.	Total MXN	Descuento	CONFIRMABLE	Nota
O001	sí	sí	no	4	2100.00	10 %	sí	E01
O002	no	sí	no	2	1050.00	0 %	sí	E02
O003	sí	no	no	4	2100.00	10 %	no	E03: cliente inactivo
O005	sí	sí	sí	4	2100.00	0 %	sí	E04: bloqueado por promoción
O006	no	sí	no	1	18000.00	5 %	sí	E05
O007	sí	sí	no	0	0.00	0 %	no	E06: pedido vacío
O008	no	sí	no	20	2400.00	0 %	sí	E07: 20 unidades permitidas
O010	no	sí	no	2	500.00	0 %	sí	E09: 500 no supera `>500`
O011	sí	sí	no	3	1500.00	0 %	sí	E10: solo 3 unidades

O009 (E08) carga 19 unidades y recibiría 10 %. El rechazo de E08 ocurre al agregar 2 unidades más, no al evaluar una Specification.

Evidencia de las evaluaciones

```
E01 O001 — evaluar_descuento (esperado: 10%)
   inicial: DRAFT · 4 u · 2100.00 MXN  →  final: DRAFT
   resultado: evaluar: 10% (total 2100.00, descuento 210.00)
   eventos: ninguno
```

Figura 2. *E01: O001 recibe 10 % (VIP, total 2100.00, 4 unidades, sin promoción).*

```
E04 O005 — evaluar_descuento (esperado: 0%)
   inicial: DRAFT · 4 u · 2100.00 MXN  →  final: DRAFT
   resultado: evaluar: 0% (total 2100.00, descuento 0.00)
   eventos: ninguno
```

Figura 3. *E04: O005 recibe 0 % porque la promoción activa bloquea los descuentos.*

```
E05 O006 — evaluar_descuento (esperado: 5%)
   inicial: DRAFT · 1 u · 18000.00 MXN  →  final: DRAFT
   resultado: evaluar: 5% (total 18000.00, descuento 900.00)
   eventos: ninguno
```

Figura 4. *E05: O006 recibe 5 % (total 18000.00 sin promoción; no es VIP).*

```
E09 O010 — evaluar_descuento (esperado: 0%)
   inicial: DRAFT · 2 u · 500.00 MXN  →  final: DRAFT
   resultado: evaluar: 0% (total 500.00, descuento 0.00)
   eventos: ninguno
```

Figura 5. *E09: O010 recibe 0 % porque 500.00 no supera el umbral estricto de 500.*

8. Máquina de estados y Domain Events

El pedido tiene 6 estados y 5 transiciones válidas; las otras 25 combinaciones de estado y operación se rechazan sin cambiar el estado ni agregar eventos.

Estados y transiciones



Figura 6. *Máquina de estados del pedido: 6 estados y 5 transiciones.*

Solo CONFIRMED puede cancelarse y solo SHIPPED puede entregarse o devolverse; DELIVERED, CANCELLED y RETURNED no tienen salidas.

Transiciones válidas

Cada método de `Order` valida primero, cambia `status` después y registra el evento al final.

Desde	Operación	Valida antes de cambiar	Hacia	Evento (`detail`)
DRAFT	`confirm()`	Tiene artículos y el cliente está activo	CONFIRMED	`OrderConfirmed`
CONFIRMED	`ship(guía)`	La guía no está vacía	SHIPPED	`OrderShipped` (la guía)
CONFIRMED	`cancel(motivo)`	El motivo no está vacío	CANCELLED	`OrderCancelled` (el motivo)
SHIPPED	`deliver()`	—	DELIVERED	`OrderDelivered`
SHIPPED	`return_order()`	—	RETURNED	`OrderReturned`

Comprobación de la matriz completa

Se ejecutaron las 30 combinaciones de estado y operación sobre un pedido real. Hay 5 transiciones válidas y 25 prohibidas; en las 25 el estado queda igual, no se agrega ningún evento y se lanza `InvalidOrderTransition` (HTTP 409).

Estado actual	confirm	ship	cancel	deliver	return_order
DRAFT	CONFIRMED	409	409	409	409
CONFIRMED	409	SHIPPED	CANCELLED	409	409
SHIPPED	409	409	409	DELIVERED	RETURNED
DELIVERED	409	409	409	409	409
CANCELLED	409	409	409	409	409
RETURNED	409	409	409	409	409

Domain Events

`DomainEvent` es inmutable (`frozen=True`) y su nombre está en pasado, porque describe un hecho que ya ocurrió. Sus campos son `name`, `order_id`, `customer_id`, `total`, `detail` y `occurred_at` (UTC).

El orden validar → cambiar `status` → registrar el evento evita anunciar un hecho que no ocurrió: si una validación falla, se lanza la excepción antes de tocar el estado y `events` no cambia.

Los escenarios E11 a E15 ejercitan estas transiciones y se detallan en la bitácora de la sección 9. Como ejemplo, E12 confirma dos veces el pedido O013: la primera llamada registra `OrderConfirmed` y la segunda lanza `InvalidOrderTransition` sin agregar otro evento.

```
E12 O013 — confirmar dos veces (esperado: rechazo segunda)
   inicial: DRAFT · 1 u · 600.00 MXN  →  final: CONFIRMED
   resultado: confirm() #1: OK | confirm() #2: RECHAZO InvalidOrderTransitio
n (Solo DRAFT puede confirmarse (estado actual: CONFIRMED))
   eventos: OrderConfirmed
```

Figura 7. *E12: la segunda confirmación de O013 se rechaza y solo queda un evento OrderConfirmed.*

9. Ejecución de los 15 escenarios

Resultados obtenidos al ejecutar cada escenario sobre los pedidos cargados desde los CSV, sin modificar los datos. En la columna de resultado se indica el estado final y el evento emitido, o la ausencia de evento cuando la operación falló.

Caso	Estado inicial	Operación	Resultado real	Explicación
E01	O001 · DRAFT · 4 u · 2,100.00 MXN	evaluar_descuento (esperado: 10 %)	Descuento 10 % (total 2,100.00; descuento 210.00) **Estado final:** DRAFT **Evento:** Ninguno (solo consulta)	Cumple las cuatro condiciones de la regla del 10 %: VIP, total mayor a 500, al menos 4 unidades y sin promoción.
E02	O002 · DRAFT · 2 u · 1,050.00 MXN	evaluar_descuento (esperado: 0 %)	Descuento 0 % (total 1,050.00; descuento 0.00) **Estado final:** DRAFT **Evento:** Ninguno (solo consulta)	No es VIP y el total no supera 5,000, así que no cumple ninguna de las dos reglas.
E03	O003 · DRAFT · 4 u · 2,100.00 MXN	confirm() (esperado: rechazo)	**RECHAZO** InvalidOrderData: El cliente está inactivo **Estado final:** DRAFT **Evento:** Ninguno (falló la validación)	El cliente C003 está inactivo. confirm() valida antes de cambiar el estado, por eso el pedido sigue en DRAFT y no se registra evento.
E04	O005 · DRAFT · 4 u · 2,100.00 MXN	evaluar_descuento (esperado: 0 %)	Descuento 0 % (total 2,100.00; descuento 0.00) **Estado final:** DRAFT **Evento:** Ninguno (solo consulta)	La promoción activa bloquea ambos descuentos: falla la condición NOT promoción.
E05	O006 · DRAFT · 1 u · 18,000.00 MXN	evaluar_descuento (esperado: 5 %)	Descuento 5 % (total 18,000.00; descuento 900.00) **Estado final:** DRAFT **Evento:** Ninguno (solo consulta)	El total supera 5,000 y no hay promoción. No es VIP, por eso no aplica el 10 % y se usa la regla alternativa del 5 %.
E06	O007 · DRAFT · 0 u · 0.00 MXN	confirm() (esperado: rechazo)	**RECHAZO** InvalidOrderData: No se puede confirmar un pedido sin artículos **Estado final:** DRAFT **Evento:** Ninguno (falló la validación)	Un pedido vacío no puede confirmarse. El estado no cambia y no se emite OrderConfirmed.
E07	O008 · DRAFT · 20 u · 2,400.00 MXN	confirm() (esperado: éxito)	OK **Estado final:** CONFIRMED **Evento:** OrderConfirmed	20 unidades exactas están permitidas (el límite se excede con más de 20). El pedido tiene artículos y el cliente está activo.
E08	O009 · DRAFT · 19 u · 2,280.00 MXN	add_item(SKU003 × 2) (esperado: rechazo)	**RECHAZO** OrderItemLimitExceeded: Máximo 20 unidades: hay 19 y se intentan agregar 2 **Estado final:** DRAFT **Evento:** Ninguno (falló la validación)	19 + 2 = 21 unidades, más del máximo. add_item() detiene la operación antes de modificar la lista, así que el pedido conserva sus 19 unidades.
E09	O010 · DRAFT · 2 u · 500.00 MXN	evaluar_descuento (esperado: 0 %)	Descuento 0 % (total 500.00; descuento 0.00) **Estado final:** DRAFT **Evento:** Ninguno (solo consulta)	El umbral es estricto: 500 no es mayor a 500. Además el cliente no es VIP y el pedido tiene solo 2 unidades.
E10	O011 · DRAFT · 3 u · 1,500.00 MXN	evaluar_descuento (esperado: 0 %)	Descuento 0 % (total 1,500.00; descuento 0.00) **Estado final:** DRAFT **Evento:** Ninguno (solo consulta)	Es VIP y el total supera 500, pero tiene solo 3 unidades y la regla exige al menos 4.
E11	O012 · DRAFT · 1 u · 850.00 MXN	confirm() y ship("") (esperado: rechazo del envío)	confirm(): OK ship(""): **RECHAZO** InvalidOrderData: El número de guía es obligatorio **Estado final:** CONFIRMED **Evento:** OrderConfirmed (el envío falló: no se emite OrderShipped)	La confirmación es válida, pero enviar exige una guía no vacía. El pedido queda en CONFIRMED.

Caso	Estado inicial	Operación	Resultado real	Explicación
E12	O013 · DRAFT · 1 u · 600.00 MXN	confirm() dos veces (esperado: rechazo de la segunda)	confirm() #1: OK confirm() #2: **RECHAZO** InvalidOrderTransition: Solo DRAFT puede confirmarse (estado actual: CONFIRMED) **Estado final:** CONFIRMED **Evento:** OrderConfirmed (uno solo; la 2.ª llamada no emite evento)	Solo un pedido en DRAFT puede confirmarse. La segunda llamada falla sin cambiar el estado ni duplicar el evento.
E13	O014 · DRAFT · 1 u · 600.00 MXN	confirm(), ship("GUIA-014") y cancel("Cliente desistió") (esperado: rechazo de la cancelación)	confirm(): OK ship("GUIA-014"): OK cancel("Cliente desistió"): **RECHAZO** InvalidOrderTransition: Solo CONFIRMED puede cancelarse (estado actual: SHIPPED) **Estado final:** SHIPPED **Evento:** OrderConfirmed, OrderShipped (cancel falló: no se emite OrderCancelled)	Un pedido ya enviado no puede cancelarse; cancelar solo es válido desde CONFIRMED.
E14	O015 · DRAFT · 1 u · 450.00 MXN	confirm(), ship("GUIA-015") y return_order() (esperado: éxito)	confirm(): OK ship("GUIA-015"): OK return_order(): OK **Estado final:** RETURNED **Evento:** OrderConfirmed, OrderShipped, OrderReturned	La devolución es válida porque el pedido está en SHIPPED. RETURNED es un estado final.
E15	O016 · DRAFT · 1 u · 2,100.00 MXN	confirm(), ship("GUIA-016") y deliver() (esperado: éxito)	confirm(): OK ship("GUIA-016"): OK deliver(): OK **Estado final:** DELIVERED **Evento:** OrderConfirmed, OrderShipped, OrderDelivered	La entrega es válida porque el pedido está en SHIPPED. DELIVERED es un estado final.

10. API REST con FastAPI

La API expone 10 endpoints, traduce las excepciones de dominio a 404, 409 y 422, y sus pruebas pasan.

Diseño

`app/api.py` carga los pedidos en un diccionario en memoria. Cada endpoint busca el pedido, llama al método de `Order` y devuelve un DTO, sin repetir validaciones de negocio. Los errores de dominio se traducen en un solo manejador global (`@app.exception_handler(DomainException)`), en lugar de un `try/except` por endpoint.

Excepción de dominio	HTTP
`InvalidOrderTransition`	409
`InvalidOrderData`	422
`OrderItemLimitExceeded`	422
Otra `DomainException`	400
Pedido, cliente o SKU inexistente (`HTTPException`)	404

Endpoints

Las operaciones que cambian el estado son `POST`, como pide la guía. `POST /orders`, `POST /orders/{id}/items` y `POST /reset` son ampliaciones. Swagger queda en `/docs` y el contrato en `/openapi.json`.

Método y ruta	Qué hace	Tag
`GET /orders`	Lista los pedidos, con descuento y eventos	consultas
`GET /orders/{id}`	Detalle de un pedido	consultas
`POST /orders`	Crea un pedido (201)	pedidos
`POST /orders/{id}/items`	Agrega un artículo	pedidos
`POST /orders/{id}/confirm`	DRAFT → CONFIRMED	transiciones
`POST /orders/{id}/ship`	CONFIRMED → SHIPPED (body: `tracking_number`)	transiciones
`POST /orders/{id}/cancel`	CONFIRMED → CANCELLED (body: `reason`)	transiciones
`POST /orders/{id}/deliver`	SHIPPED → DELIVERED	transiciones
`POST /orders/{id}/return`	SHIPPED → RETURNED	transiciones
`POST /reset`	Recarga los 16 pedidos desde los CSV	práctica

Respuestas reales

Obtenidas al ejecutar la API con `TestClient` sobre los datos del paquete. Cubren los tres casos que pide la guía para Swagger: ID inexistente (404), doble confirmación (409) y pedido sin artículos (422).

Petición	HTTP	Cuerpo
`GET /orders/O001`	200	DRAFT, 4 unidades, total 2100.00
`GET /orders/NOEXISTE`	404	`No existe el pedido NOEXISTE`
`POST /orders/O001/confirm`	200	CONFIRMED, evento `OrderConfirmed`
Mismo `confirm`, 2.ª vez	409	`Solo DRAFT puede confirmarse (estado actual: CONFIRMED)`
`POST /orders/O007/confirm` (vacío)	422	`No se puede confirmar un pedido sin artículos`
`POST /orders/O003/confirm` (inactivo)	422	`El cliente está inactivo`
`POST /orders/O012/ship` con guía vacía	422	`El número de guía es obligatorio`
`POST /orders/O012/ship` con `G1`	200	SHIPPED
`POST /orders/O012/cancel` desde SHIPPED	409	`Solo CONFIRMED puede cancelarse (estado actual: SHIPPED)`
`POST /orders/O009/items` SKU003 ×2	422	`Máximo 20 unidades: hay 19 y se intentan agregar 2`
Artículo con cantidad 0	422	`cantidad debe ser mayor a 0`
`POST /orders` con ID repetido	409	`El pedido O100 ya existe`

Evidencia en Swagger

El servidor se levantó con `uvicorn app.api:app --reload` y la documentación interactiva se abrió en `/docs`. La ruta raíz `/` devuelve 404 porque la API no define una; es un comportamiento esperado.

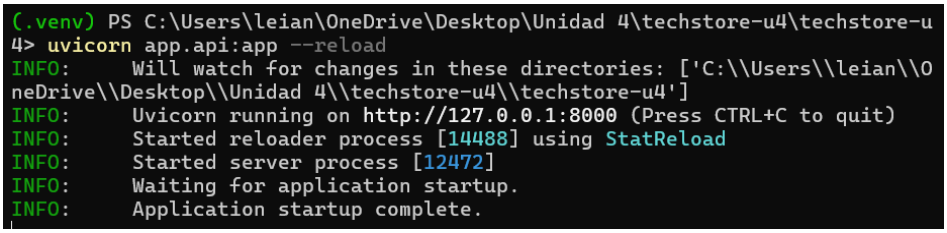


Figura 8. *Servidor levantado con uvicorn (Application startup complete).*

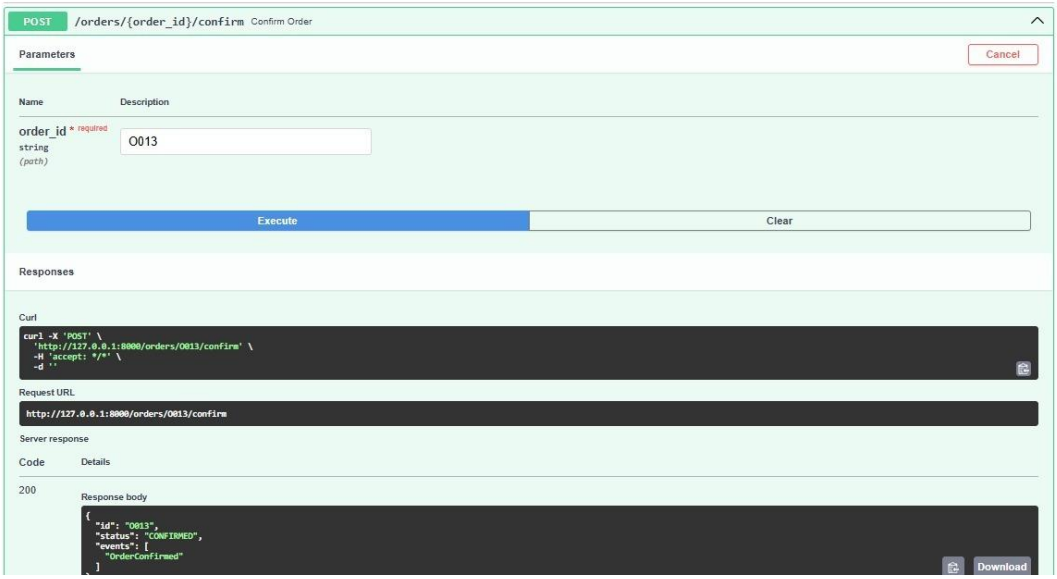


Figura 9. *Operación exitosa: confirmar el pedido O013 devuelve 200 y el evento OrderConfirmed.*

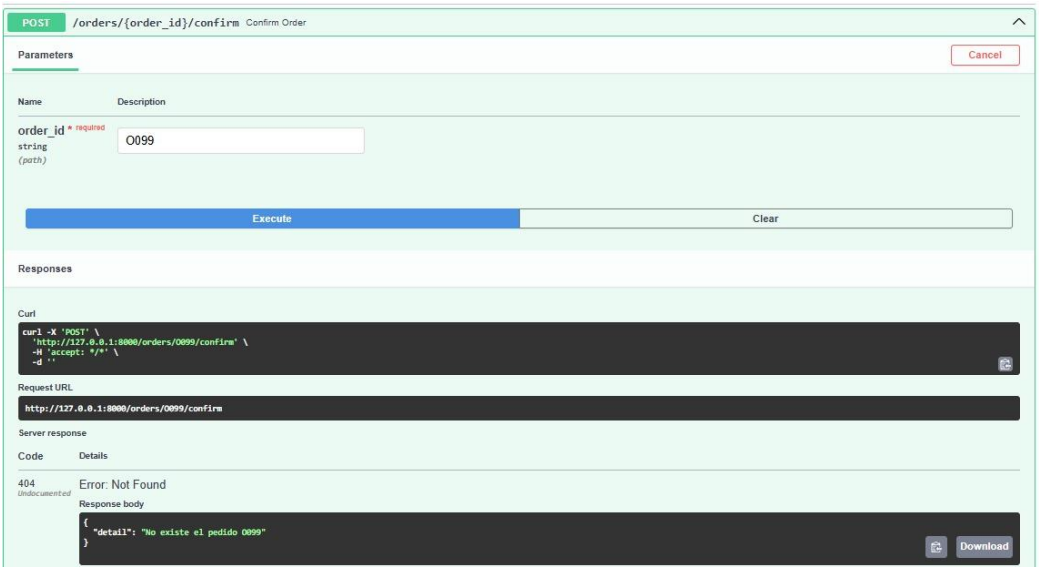


Figura 10. *Operación rechazada: confirmar un ID inexistente (O099) devuelve 404.*

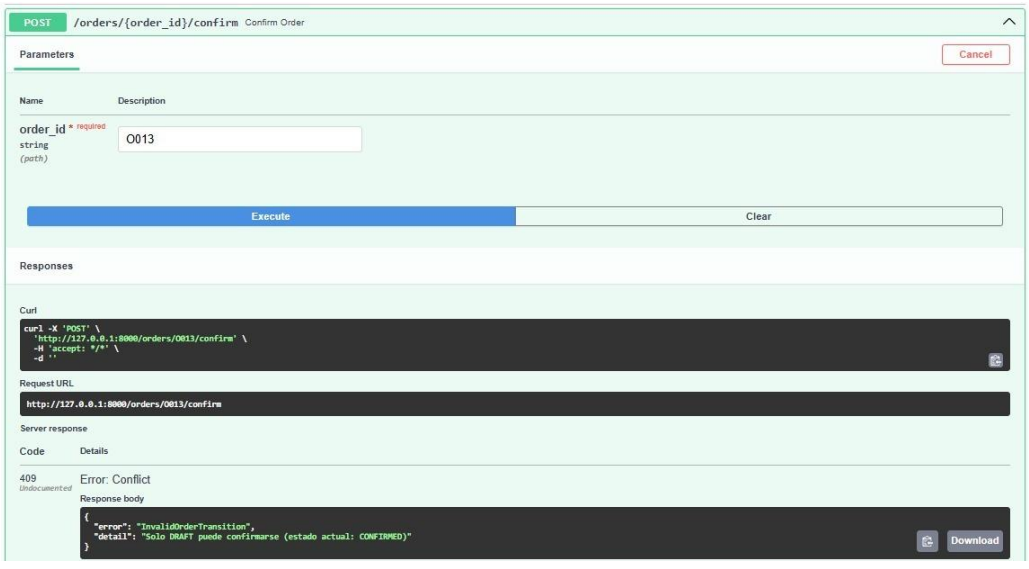


Figura 11. *Operación rechazada: confirmar dos veces el pedido O013 devuelve 409 (InvalidOrderTransition).*

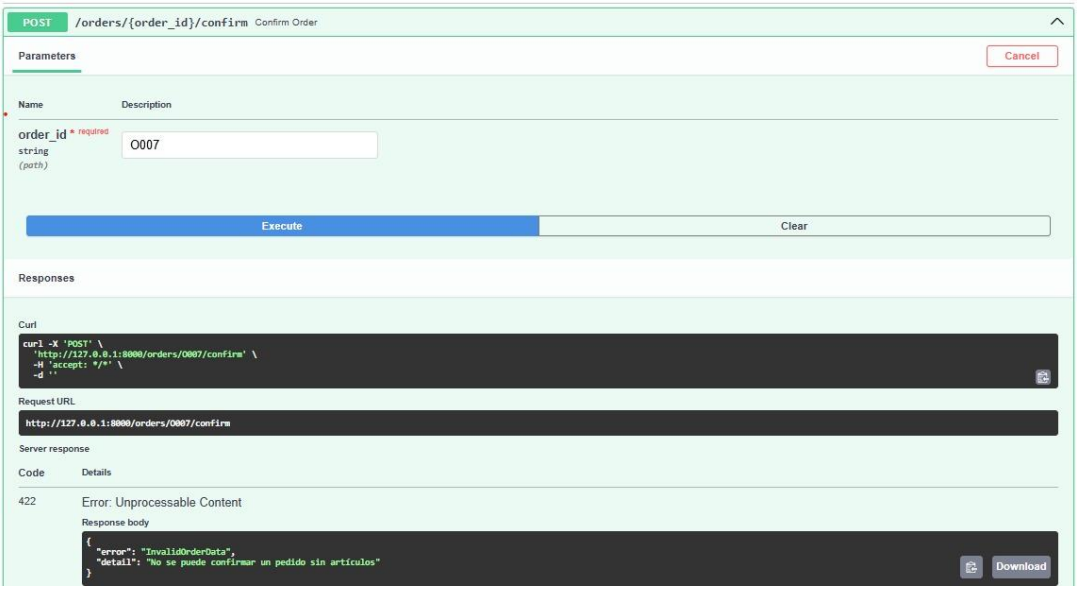


Figura 12. *Operación rechazada: confirmar el pedido vacío O007 devuelve 422 (InvalidOrderData).*

11. Pruebas con pytest

`python -m pytest -q` termina con 51 pruebas aprobadas (41 de dominio y 10 de API) y ninguna fallida. Las pruebas cubren los casos que pide la guía:

Caso a demostrar	Prueba
21 unidades (límite de 20)	test_e08_veintiuna_unidades_se_rechazan_sin_modificar
Cliente inactivo	test_e03_cliente_inactivo_no_confirma
Pedido vacío	test_e06_pedido_vacio_no_confirma
Descuento VIP y promoción activa	test_descuentos_escenarios
Envío sin guía	test_e11_envio_sin_guia
Confirmación doble	test_e12_confirmacion_doble
Cancelación desde SHIPPED	test_e13_no_se_cancela_desde_shipped
Respuesta HTTP 404	test_404_pedido_inexistente

La captura muestra 1 advertencia (`StarletteDeprecationWarning`): el `TestClient` de Starlette avisa que usar `httpx` está obsoleto y sugiere instalar `httpx2`. Es una advertencia de una librería, no un fallo, y no afecta los resultados.

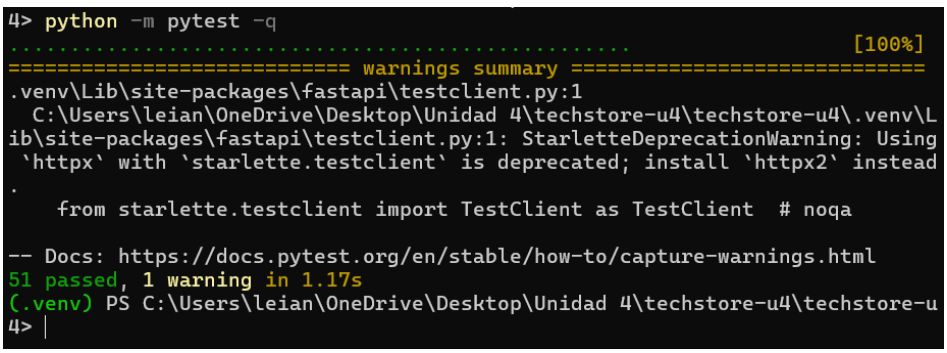


Figura 13. *Resultado de python -m pytest -q: 51 passed, 1 warning.*

12. Lista de verificación antes de entregar

	Punto de verificación	Evidencia
☑	Puedo ejecutar python cargar_csv.py sin errores.	Sección 6, Figura 1: los 16 pedidos cargan en DRAFT.
☑	O001 muestra 4 unidades y 2100.00 MXN.	Sección 6 (Figura 1) y sección 4 (cálculo manual).
☑	La regla del 10 % requiere VIP, >500, >=4 unidades y ausencia de promoción.	Sección 7: TEN_PERCENT; Figuras 2 y 3 (E01 y E04).
☑	La regla del 5 % requiere >5000 y ausencia de promoción.	Sección 7: FIVE_PERCENT; Figura 4 (E05).
☑	Una operación inválida no cambia el estado ni añade eventos.	Sección 8 (matriz de 25 transiciones rechazadas) y sección 9 (E03, E06, E08, E11, E12, E13).
☑	La API responde 404, 409 y 422 en los casos indicados.	Sección 10: Figuras 10 (404), 11 (409) y 12 (422); Figura 9 muestra la operación exitosa (200).
☑	pytest termina sin fallos.	Sección 11, Figura 13: 51 passed.
☐	El repositorio contiene README y evidencias.	Pendiente: el proyecto ya tiene README.md y las capturas están en este reporte; falta subirlos a GitHub y anotar el enlace del repositorio.

13. Preguntas para el reporte

¿Por qué se usa Decimal en lugar de float para precios?

Un `float` guarda los números en binario y no representa exactamente muchos decimales: `0.1 + 0.2` da `0.30000000000000004`. Con dinero, esos errores se acumulan en sumas y porcentajes. `Decimal` hace aritmética decimal exacta, siempre que se construya desde texto (`Decimal("600.00")`), porque `Decimal(600.00)` hereda la imprecisión del float. Por eso `Money` rechaza los montos que no sean `Decimal`.

¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es un producto distinto dentro del pedido; las unidades son la suma de sus cantidades. El pedido O008 tiene 1 renglón y 20 unidades. El límite de 20 es de unidades: si se contaran renglones, O008 pasaría como "1" y se podrían agregar cientos de piezas. Por eso `Order.units` usa `sum(item.quantity ...)`.

¿Qué ventaja tiene encapsular una regla en Specification?

La regla tiene nombre y se prueba de forma aislada (`VIPCustomerSpec`, `MinimumUnitsSpec`). Se combina con `&`, `|` y `~` sin repetir `if` en varias partes del código y se reutiliza en la API, en un proceso batch o en las pruebas. Si el negocio cambia el umbral de 500 a 800, se cambia en un solo lugar. Además, la Specification solo evalúa y no modifica el pedido.

¿Por qué los eventos se registran solo después de una transición válida?

Un evento afirma que algo ocurrió, y otros sistemas pueden reaccionar a él: enviar un correo, facturar o reservar inventario. Si se registrara antes de validar y la operación fallara, se anunciaría un hecho falso. El orden validar, cambiar el estado y registrar el evento garantiza que cada evento corresponde a un cambio real; en E12, por ejemplo, la segunda confirmación falla y `events` conserva un solo `OrderConfirmed`.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Todo `app/domain.py` y `app/specifications.py`, además de `cargar_csv.py` y las pruebas de dominio, porque ninguno importa FastAPI. Solo cambiaría `app/api.py`, que se limita a buscar el

pedido, llamar al método de `Order` y traducir las excepciones a HTTP. El mismo núcleo serviría para una consola, una interfaz gráfica o un proceso batch.

14. Reflexiones

Durante la práctica encontramos tres errores principales. El primero fue que pytest fallaba porque el entorno virtual no tenía FastAPI instalado; lo resolvimos instalando las dependencias de requirements.txt dentro de .venv. El segundo fue que el caso E09 no demostraba por sí solo que el umbral de 500 fuera estricto, porque el pedido fallaba la regla del 10 % por otras razones; lo corregimos con una prueba que usa un pedido VIP de exactamente 500 y 4 unidades. El tercero fue interpretar la advertencia de pytest y el 404 de la ruta raíz como fallos, cuando en realidad se trataba de una advertencia de una librería y de un comportamiento esperado.

Aprendimos que las reglas de negocio deben vivir en el dominio y no en la API, que una prueba solo vale si falla por la razón correcta y que los eventos son confiables solo si se registran después de una transición válida.

15. Enlace GitHub:

https://github.com/emir-lopez/TECH_STORE_E8.git